FAST FUNCTIONAL UPDATES: TYPE DISCIPLINE FOR UNOBSERVABLE UPDATES

How a type discipline makes a write correct, and how that becomes a compiler optimisation on reference count one

FAST FUNCTIONAL UPDATES: TYPE DISCIPLINE FOR UNOBSERVABLE UPDATES

PURE FUNCTIONAL PROGRAMMING

A pure functional programming language cannot mutate.

This is not merely an inconvenience: Certain algorithms can only be low complexity in a language with in-place updates (See: Union Find).

But there is a trick: A compiler can update `in place` when it doesn't change what a program `means`.

We want to show you how we can get *unobservable updates* in easy steps:

> `type discipline` → writes are `unobservable` → an in-place update is `correct`;
>
> A compiler can emit a program where a ==reference count=1== yields an in-place update.
>
> A type discipline can `ensure` that reference count when we mean it.

THE PROBLEM IN A NUTSHELL

Lean has no mutable state but we sometimes pretend it does. For instance `a.set! i v` is a function. This, mathematically speaking, refers to two structures: the one before and the one after.

Nothing in a theorem prover (or functional language) can change a value that already exists or it breaks the maths.

Copying is therefore the price of purity: schematically this means a whole-object allocation for every single "update".

This is clearly insane if you are trying to sort an array.

This is unfair! We want speed. We want what all imperative programmers have. We want `mutation`.

Can we have our cake and eat it too? (think about how literally this aphorism applies!)

YES: MAKE THE WRITE UNOBSERVABLE

Unobservable? That doesn't sound very helpful! What does unobservable mean?

Nobody knows that we're *cheating* and doing an in-place update because they can't see the original value anymore.

The motto: An update in place is correct exactly when it cannot be observed.

If the old cell has no other reference, then "the same object with one slot changed" and "a new object, copied" are indistinguishable.

A write becomes legal exactly when it is unobservable.

REALISED AS A COMPILER OPTIMISATION

In fact, the `Lean` compiler already does this.

At every store operation it asks the question: `is this object's reference count exactly 1 here?`

> count `1` → the old value is dead, store on top of it — an in-place mutation.
>
> count `2+` → others are looking, someone may still read the old value — we have to allocate a copy.
>
> count `0` → this is a compile-time constant — this is never unique, always copy.

The backend mechanism already exists. `Lean` compiles away the copy `whenever it can see` the count is one.

THE RUB: CAN WE ENSURE RC = 1?

This strategy kicks off only if the count is 1.

Is there a way to make the count 1?

We want a static property of the program source.

A type discipline which can ensure we will always do our update when RC=1.

We don't want to *think* we are using an update only to find we have a copy!

This would break our ability to reason about performance.

The compiler's mutation should be both correct (unobservable) and real (performed).

THE TYPE DISCIPLINE: OWNERSHIP

The discipline is **ownership** (Rust fans shout hurray!).

A write goes through a cell the program holds **exactly once**.

A written name is used **once** in the scope that binds it.

This name is bound to a **construction**.

The shapes where a **second holder** exists are **refused**.

These refused shapes are: duplication, an alias, a use in a sibling position.

../LEAN/SCIDONIA/RC/OWNERSHIPTYPED/JUDGEMENT.LEAN

```
/-- A write is admitted only at the owner the subject names, on the subject's
own name and at index zero. -/
| write {Γ : TyEnv} (x : Var) {a : Expr} (hx : Γ x = some .cell)
    (ha : Obs Γ a slotObs) : Own Γ (.held x) (.store (.var x) 0 a) .cell
```

REVISITING THE REASONING CHAIN

A `Typed program` ⇒ the write's cell is `uniquely held` (the ownership rule).

⇒ under a `correspondence`, the runtime `count is 1` (`bridge`).

⇒ therefore the write is `destructive` (`store_owned_is_destructive`).

⇒ *And* it is `unobservable`: i.e. the in-place run and the copy run are the `same value` (`evalArray_sound`).

../LEAN/SCIDONIA/RC/CORRESPONDENCE.LEAN

```
theorem bridge {ρ : TEnv} {m : Multi} {Γ : Occs} {y : Var} {r : Ref} {i : Nat} {v : Expr}
    (h : Corr ρ m Γ) (hlook : ρ.lookup y = some r) (hone : Γ.count y = 1) :
    Owns ρ m (.store (.var y) i v) ∧ (expand m).count r = 1 := by
```

COUNT ONE ⇒ DESTRUCTIVE

When the account counts the subject's reference **exactly once** — and no heap slot, no run handle names the cell — the write keeps the cell's location.

A store at any other count **copies**: the written slots land at a fresh location.

../LEAN/SCIDONIA/RC/OWNERSHIP.LEAN

```
theorem store_owned_is_destructive {c : CHeap} {hs : List Ref} {ρ : TEnv} {m : Multi}
    {y : Var} {r : Ref} {i : Nat} {v : Expr} {ks : List Ref} (hc : Couple c hs ρ m)
    (hρ : ρ.lookup y = some r) (hown : Owns ρ m (.store (.var y) i v))
    (hm : (expand m).count r = 1) (hslot : (CHeap.slots c).count r = 0)
    (hrun : hs.count r = 0) (hks : r ∉ ks) :
    (update c (hs ++ expand m) r ks).1.lookup r = some ks := by
```

AND IT IS UNOBSERVABLE

The sealed evaluation and the copying reduction of the same term reach exactly the same value.

That is the payoff: the update is not just correct in intent — it is proved equal to the pure meaning.

../LEAN/SCIDONIA/RC/OWNERSHIPTYPED/REFINEMENT.LEAN

```
theorem evalArray_sound {o : ObsDesc} {e v : Expr}
    (h : Own (TyEnv.empty : TyEnv) .none e (.obs o)) (_hc : Closed e)
    (hs : StepStar e v) (hv : Value v) : evalArray e = some v := by
```

A TREE: MUCH SHARED FOR READS, ONLY THE WRITE TOTAL

This same discipline scales to trees: a read borrows a fraction of the spine's nodes — so many readers can share one spine — and a write holds the whole of its target subtree.

That is "much of the tree shared out for reads, and only the write has to be a total fraction."

../LEAN/SCIDONIA/RC/TREEOWNERSHIP/JUDGEMENT.LEAN

```
def WriteHolds {d : Layout} (A : Account d) (w : Nat) (p : Path d) : Prop :=
  (∀ q, InSubtree p q → Writes A q) ∧ (∀ r, PrefixOf r p → r ≠ p → ∃ q, 0 < q ∧ Reads A r w
q)
```

WHAT WE HAVE SHOWN VS. WHAT WE HAVEN'T

Our proof is a claim about a model: the discipline yields a count of 1 in the λ-RC heap and the update is correct (unobservable).

We have used the idea of the following notional compile chain: L∃∀N ⇒ λ-Pure ⇒ λ-RC ⇒ C

We have proved that the type discipline in λ-Pure obtains the desired behaviour in λ-RC

We have checked experimentally and found that our discipline appears to work in the C code.

⚠ We have not shown that the λ-RC behaviour is faithfully reflected in the C

⚠ We have not shown that our type discipline in Lean will survive to λ-Pure

But it should be quite persuasive and the full chain should be possible!